

Informe LAB O1

Santiago Murillo Ibañez

Universidad Escuela Colombiana de Ingeniería Julio Garavito

Paradigmas de la Programación

Juan Camilo Ángel Hernández

1 de agosto de 2026

Ejercicio 1 - Hechos y consultas simples

1) ¿Existe un vuelo directo de Bogotá a Medellín?

```
vuelo(bogota, medellin)
true
```

2) ¿Qué destinos se pueden alcanzar directamente desde Bogotá?

```
vuelo(bogota, X)
X = medellin
X = cartagena
X = pasto
```

3) ¿Desde que destinos se puede alcanzar Medellín?

```
vuelo(X, medellin)
X = bogota
```

4) ¿Hay alguna forma de llegar directamente a Cali?

```
vuelo(X, cali)
false
```

Ejercicio 2 - Reglas básicas.

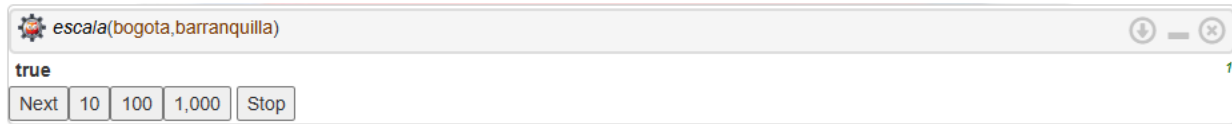
Foto 1

```
1 % Hechos: relación entre ciudades
2 ciudad(bogota).
3 ciudad(medellin).
4 ciudad(cali).
5 ciudad(cartagena).
6 ciudad(manizales).
7 ciudad(barranquilla).
8 ciudad(pasto).
9 ciudad(monteria).
10
11 % Hechos: vuelos directos
12 vuelo(bogota, medellin).
13 vuelo(medellin, cartagena).
14 vuelo(cali, bogota).
15 vuelo(bogota, cartagena).
16 vuelo(manizales, cartagena).
17 vuelo(medellin, barranquilla).
18 vuelo(pasto, bogota).
19 vuelo(bogota, pasto).
20
21 escala(X,Y) :- vuelo(X,Z), vuelo(Z,Y).
22 viaje(X,Y) :- escala(X,Y) ; vuelo(X, Y).
23 destinos(X,Y) :- vuelo(X,Y).
```

1) Determine si dos ciudades están conectadas mediante una escala.

Foto 1 o archivo

Ahora, verifica si existe una conexión de Bogotá a Barranquilla.



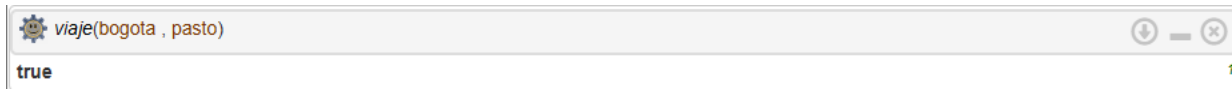
2) Encuentra todas las ciudades a las que se puede llegar desde Cali con una escala.



3) Define una regla viaje que sea cierta si existe un vuelo directo o con una escala entre dos ciudades (no usar recursión).

Foto 1 o archivo

Ahora, verifica si existe un viaje posible de Bogotá a Pasto.



4) Define una regla destinos que devuelva todos los destinos alcanzables directamente desde una ciudad.

Foto 1 o Archivo

Foto 2

```
perro(firulaís).
perro(bruno).
perro(max).
gato(misu).
gato(luna).
gato(chanel).
gato(orion).
ave(piolin).

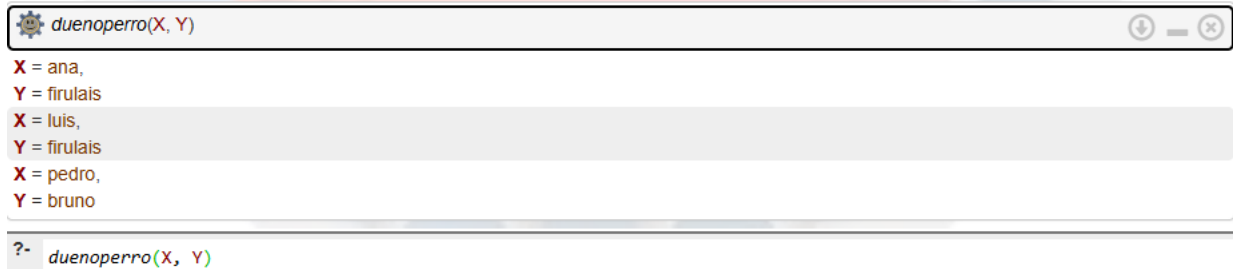
dueno(ana, firulaís).
dueno(ana, misu).
dueno(luis, luna).
dueno(luis, orion).
dueno(luis, firulaís).
dueno(maria, piolin).
dueno(julia, chanel).
dueno(pedro, bruno).

duenoperro(X,Y) :- dueno(X,Y), perro(Y).
duenogato(X,Y) :- dueno(X,Y), gato(Y).
duenomultiples(X) :- dueno(X,Y), dueno(X,Z), Y\=Z.
amante_animales(X) :- dueno(X,Y), perro(Y), dueno(X,Z), gato(Z).
mascota_compartida(X,Y) :- dueno(X,Z),dueno(Y,A), Z==A.
tipo_mascota(X, perro) :- dueno(X, Y), perro(Y).
tipo_mascota(X, gato) :- dueno(X, Y), gato(Y).
tipo_mascota(X, ave) :- dueno(X, Y), ave(Y).
```

5) Define una regla que determine si una persona tiene un perro.

Foto 2 o Archivo

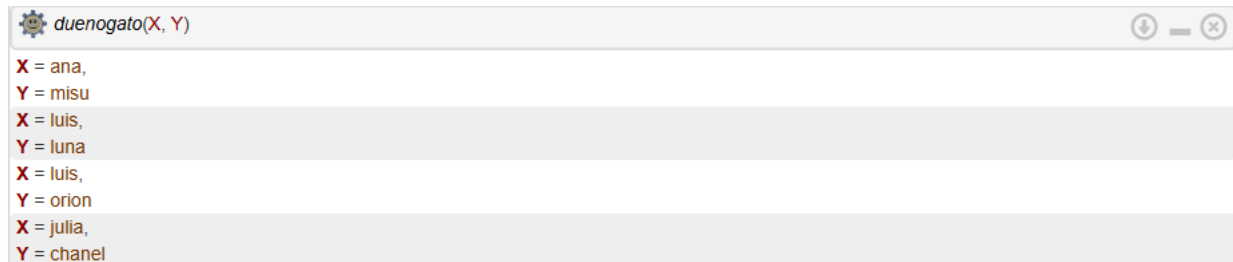
Ahora, encuentra los dueños de perros.



6) Define una regla que determine si una persona tiene un gato.

Foto 2 o Archivo

Ahora, encuentra los dueños de gatos.



7) Define una regla que determine si una persona tiene múltiples mascotas.

Foto 2 o Archivo

Ahora, encuentra los dueños de multiple tipos de mascota.



8) Define una regla amante_animales para identificar dueños que tienen tanto perro como gato.

Foto 2 o Archivo

- 9) Define una regla mascota_compartida que indique si dos personas comparten mascota.

Foto 2 o Archivo

- 10) Define una regla tipo_mascota que asocie una persona con el tipo de mascota que tiene (perro, gato, ave, etc.).

Foto 2 o Archivo

Ejercicio 3 - Predicados con condiciones aritméticas

Foto 3

```
% Hechos: estudiante y su nota final en un curso
nota(ana, 4.5).
nota(luis, 2.8).
nota(maria, 3.7).
nota(juan, 5.0).
nota(pedro, 2.3).

reprueba(X) :- nota(X,Y), 3.0>Y.
aprueba(X) :- nota(X,Y), Y>3.0.
rango(X,Y,Z) :- nota(X,N), Z>=N, N>=Y.
clasificacion(X,Y) :- nota(X,N), N>0.0, 2.9>=N, Y=reprobado.
clasificacion(X,Y) :- nota(X,N), N>2.9, 3.9>=N, Y=aprobado.
clasificacion(X,Y) :- nota(X,N), N>3.9, 4.4>=N, Y=notable.
clasificacion(X,Y) :- nota(X,N), N>4.4, 5>=N, Y=excelente.
```

- 1) Define una regla reprueba, que sea cierta si un estudiante tiene menor a 3.0.

Foto 3 o Archivo

- 2) Define una regla aprueba, que sea cierta si un estudiante tiene nota mayor o igual a 3.0.

Foto 3 o Archivo

- 3) Define una regla rango, donde rango(X, Min, Max) sea cierto si la nota de un estudiante X está entre Min y Max.

Foto 3 o Archivo

Foto 4

```

distancia((X1, Y1), (X2, Y2), D) :- Dx is X2 - X1, Dy is Y2 - Y1, D is sqrt(Dx*Dx + Dy*Dy).

% Caso base: lista vacía o con un solo punto -> distancia 0
distancia_total([], 0).
distancia_total([_], 0).

distancia_total([(X1, Y1), (X2, Y2) | Resto], Total) :-
    distancia((X1, Y1), (X2, Y2), D), distancia_total([(X2, Y2) | Resto], TotalResto),
    Total is D + TotalResto.

```

- 4) Define la regla distancia((X1, Y1), (X2, Y2), D), que calcule la distancia euclidiana entre dos puntos.

Foto 4 o Archivo

- 5) Define la regla distancia_total([P1, P2 | Resto], Total), que calcule la suma de distancias entre una lista puntos.

Foto 4 o Archivo

Ejercicio 4 - Recursión simple

Foto 5

```

fibonacci(0, 0).
fibonacci(1, 1).

fibonacci(X, Total) :- X > 1, X1 is X-1, X2 is X-2,
    fibonacci(X1, Total1), fibonacci(X2, Total2),
    Total is Total1+Total2.

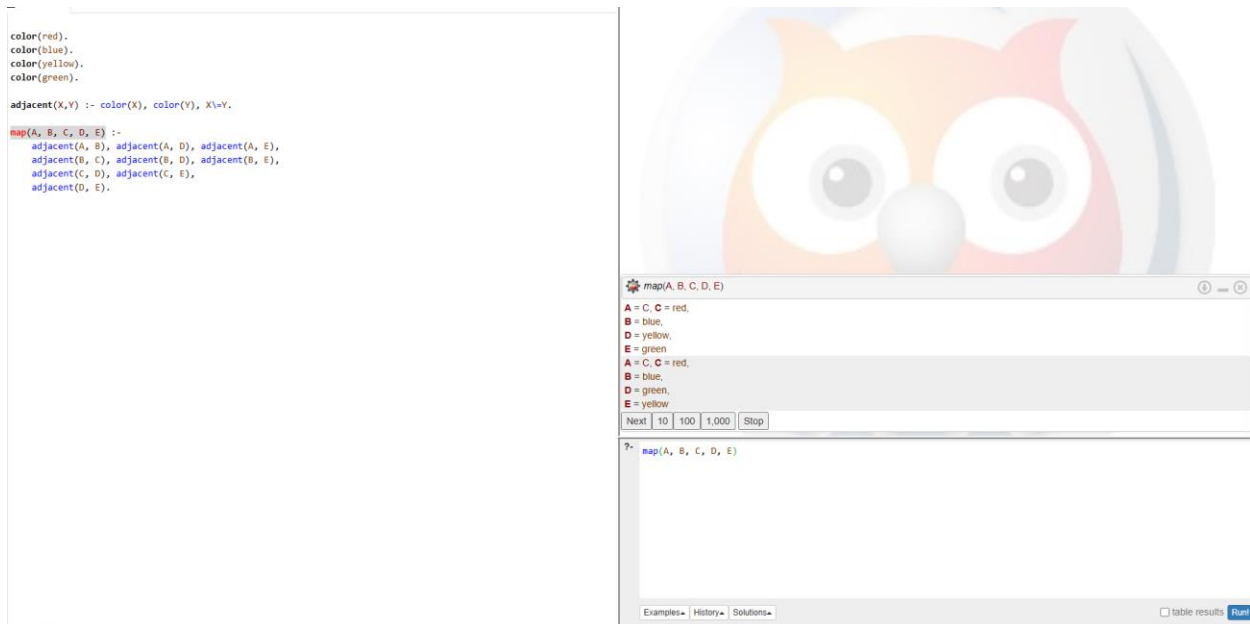
```

- 1) Define la regla fibonacci(X, N)

Foto 5 o Archivo

Ejercicio 6 – Listas

Foto 6



Faltaria definir la regla adjacent la cual valide la adyacencia valida.

Foto 6 o documento

Foto 7

```

miembro(X, [X|_]).
miembro(X, [_|T]) :- miembro(X, T).

longitud([],0).
longitud(_|T,Long) :-
    longitud(T,L1),
    Long is L1+1.

concatenar([], L, L).
concatenar([H|T1], L2, [H|T3]) :- concatenar(T1, L2, T3).

fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(X,Total) :- X>1, X1 is X-1, X2 is X-2,
    fibonacci(X1,Total1), fibonacci(X2,Total2),
    Total is Total1+Total2.

listafibo(0, []).
listafibo(N, Lista) :- N>0, N1 is N-1,
    listafibo(N1, Resto), fibonacci(N1, F), append(Resto, [F], Lista).

reverso([],[]).
reverso([H|T], R) :- reverso(T,RT), append(RT, [H], R).

palindroma(X) :- reverso(X, X).
  
```

- 1) Escribe una regla que verifique si un elemento es miembro de una lista.

Foto 7 o Archivo

- 2) Escribe una regla "longitud" que, dada una lista, calcule su longitud.

Foto 7 o Archivo

- 3) Escribe una regla "concatena" que dadas dos listas, las concatene en otra.

Foto 7 o Archivo

- 4) Escribe una regla que permite obtener en una lista los N elementos de la serie de Fibonacci, usa la regla predefinida `append(L1,L2,L3)`, que concatena L1 y L2, en L3.

Foto 7 o Archivo

- 5) Escribe una regla que determine el reverso de una lista.

Foto 7 o Archivo

- 6) Escribe una regla que determine si una lista es palíndroma.

Foto 7 o Archivo